arXiv:2305.08845v2 [cs.IR] 24 Jan 2024

Large Language Models are Zero-Shot Rankers for Recommender Systems

Yupeng Hou[1,2†], Junjie Zhang[1†], Zihan Lin[3], Hongyu Lu[4], Ruobing Xie[4], Julian McAuley[2], and Wayne Xin Zhao[1✉]

[1] Gaoling School of Artificial Intelligence, Renmin University of China
[2] UC San Diego
[3] School of Information, Renmin University of China
[4] WeChat, Tencent
yphou@ucsd.edu junjie.zhang@ruc.edu.cn batmanfly@gmail.com

Abstract. Recently, large language models (LLMs) (*e.g.,* GPT-4) have demonstrated impressive general-purpose task-solving abilities, including the potential to approach recommendation tasks. Along this line of research, this work aims to investigate the capacity of LLMs that act as the ranking model for recommender systems. We first formalize the recommendation problem as a conditional ranking task, considering sequential interaction histories as *conditions* and the items retrieved by other candidate generation models as *candidates*. To solve the ranking task by LLMs, we carefully design the prompting template and conduct extensive experiments on two widely-used datasets. We show that LLMs have promising zero-shot ranking abilities but (1) struggle to perceive the order of historical interactions, and (2) can be biased by popularity or item positions in the prompts. We demonstrate that these issues can be alleviated using specially designed prompting and bootstrapping strategies. Equipped with these insights, zero-shot LLMs can even challenge conventional recommendation models when ranking candidates are retrieved by multiple candidate generators. The code and processed datasets are available at https://github.com/RUCAIBox/LLMRank.

Keywords: Large Language Model · Recommender System.

1 Introduction

In the literature of recommender systems, most existing models are trained with user behavior data from a specific domain or scenario [49,26,28], suffering from two major issues. Firstly, it is difficult to capture user preference by solely modeling historical behaviors, *e.g.,* clicked item sequences [28,33,82], limiting the expressive power to model more complicated but explicit user interests (*e.g.,* intentions expressed in natural language). Secondly, these models are essentially

† Equal contribution.
✉ Corresponding author.

"*narrow experts*", lacking more comprehensive knowledge in solving complicated recommendation tasks that rely on background or commonsense knowledge [23].

To improve recommendation performance and interactivity, there have been increasing efforts that explore the use of pre-trained language models (PLMs) in recommender systems [21,30,62]. They aim to explicitly capture user preference in natural language [21] or transfer rich world knowledge from text corpora [30,29]. Despite their effectiveness, thoroughly fine-tuning the recommendation models on task-specific data is still a necessity, making it less capable of solving diverse recommendation tasks [30]. More recently, large language models (LLMs) have shown great potential to serve as zero-shot task solvers [64,52]. Indeed, there are some preliminary attempts that employ LLMs for solving recommendation tasks [20,59,60,13,40,74]. These studies mainly focus on discussing the possibility of building a capable recommender with LLMs. While promising, the insufficient understanding of the new characteristics when making recommendations using LLMs could hinder the development of this new paradigm.

In this paper, we conduct empirical studies to investigate what determines the capacity of LLMs that serve as recommendation models. Typically, recommender systems are developed in a pipeline architecture [10], consisting of *candidate generation* (retrieving relevant items) and *ranking* (ranking relevant items at a higher position) procedures. This work mainly focuses on the ranking stage of recommender systems, since LLMs are more expensive to run on a large-scale candidate set. Further, the ranking performance is sensitive to the retrieved candidate items, which is more suitable to examine the subtle differences in the recommendation abilities of LLMs.

To carry out this study, we first formalize the recommendation process of LLMs as a *conditional ranking* task. Given prompts that include sequential historical interactions as *"conditions"*, LLMs are instructed to rank a set of *"candidates"* (*e.g.,* items retrieved by candidate generation models), according to LLM's intrinsic knowledge. Then we conduct control experiments to systematically study the empirical performance of LLMs as rankers by designing specific configurations for "conditions" and "candidates", respectively. Overall, we attempt to answer the following key questions:

- What factors affect the zero-shot ranking performance of LLMs?
- What data or knowledge do LLMs rely on for recommendation?

Our empirical experiments are conducted on two public datasets for recommender systems. The results lead to several key findings that potentially shed light on how to develop LLMs as powerful ranking models for recommender systems. We summarize the key findings as follows:

- LLMs *struggle to perceive the order* of the given sequential interaction histories. By employing specifically designed promptings, *LLMs can be triggered to perceive the order*, leading to improved ranking performance.
- LLMs suffer from position bias and popularity bias while ranking, which can be alleviated by bootstrapping or specially designed prompting strategies.

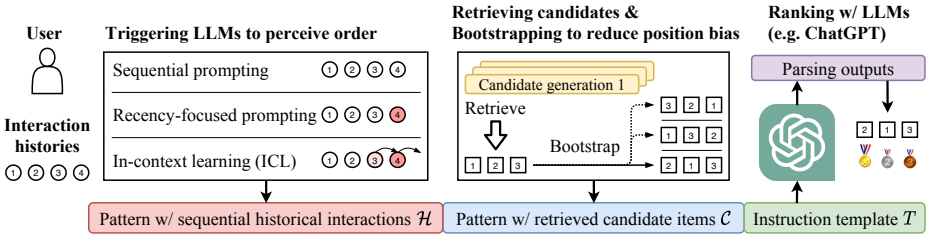

Fig. 1: An overview of the proposed LLM-based zero-shot ranking method.

– LLMs outperform existing zero-shot recommendation methods, showing promising zero-shot ranking abilities, especially on candidates retrieved by multiple candidate generation models with different practical strategies.

2 General Framework for LLMs as Rankers

To investigate the recommendation abilities of LLMs, we first formalize the recommendation process as a conditional ranking task. Then, we describe a general framework that adapts LLMs to solve the recommendation task.

2.1 Problem Formulation

Given the historical interactions $\mathcal{H} = \{i_1, i_2, \ldots, i_n\}$ of one user (in chronological order of interaction time) as *conditions*, the task is to rank the *candidate* items $\mathcal{C} = \{i_j\}_{j=1}^{m}$, such that the items of interest would be ranked at a higher position. In practice, the candidate items are usually retrieved by candidate generation models from the whole item set $\mathcal{I}$ ($m \ll |\mathcal{I}|$) [10]. Further, we assume that each item i is associated with a descriptive text t_i following [30].

2.2 Ranking with LLMs Using Natural Language Instructions

We use LLMs as ranking models to solve the above-mentioned task in an instruction-following paradigm [64]. Specifically, for each user, we first construct two natural language patterns that contain sequential interaction histories $\mathcal{H}$ (*conditions*) and retrieved candidate items $\mathcal{C}$ (*candidates*), respectively. Then these patterns are filled into a natural language template T as the final instruction. In this way, LLMs are expected to understand the instructions and output the ranking results as the instruction suggests. The overall framework of the ranking approach by LLMs is depicted in Figure 1. Next, we describe the detailed instruction design in our approach.

Sequential historical interactions. To investigate whether LLMs can capture user preferences from historical user behaviors, we include sequential historical interactions $\mathcal{H}$ into the instructions as inputs of LLMs. To enable LLMs to be

aware of the sequential nature of historical interactions, we propose three ways to construct the instructions:

- **Sequential prompting**: Arrange the historical interactions in chronological order. This way has also been used in prior studies [13]. For example, *"I've watched the following movies in the past in order: '0. Multiplicity', '1. Jurassic Park', ..."*.
- **Recency-focused prompting**: In addition to the sequential interaction records, we can add an additional sentence to emphasize the most recent interaction. For example, *"I've watched the following movies in the past in order: '0. Multiplicity', '1. Jurassic Park', ... Note that my most recently watched movie is Dead Presidents. ..."*.
- **In-context learning (ICL)**: ICL is a prominent prompting approach for LLMs to solve various tasks [79], where it includes demonstration examples in the prompt. For the personalized recommendation task, simply introducing examples of other users may introduce noises because users usually have different preferences. Instead, we introduce demonstration examples by augmenting the input interaction sequence itself. We pair the prefix of the input interaction sequence and the corresponding successor as examples. For instance, *" If I've watched the following movies in the past in order: '0. Multiplicity', '1. Jurassic Park', ..., then you should recommend Dead Presidents to me and now that I've watched Dead Presidents, then ..."*.

Retrieved candidate items. Typically, candidate items to be ranked are first retrieved by candidate generation models [10]. In this work, we consider a relatively small pool for the candidates, and keep 20 candidate items (*i.e.,* $m = 20$) for ranking. To rank these candidates with LLMs, we arrange the candidate items $\mathcal{C}$ in a sequential manner. For example, *"Now there are 20 candidate movies that I can watch next: '0. Sister Act', '1. Sunset Blvd', ..."*. Note that, following the classic candidate generation approach [10], there is no specific order for candidate items. As a result, We generate different orders for the candidate items in the prompts, which enables us to further examine whether the ranking results of LLMs are affected by the arrangement order of candidates, *i.e.,* position bias, and how to alleviate position bias via bootstrapping.

Ranking with large language models. Existing studies show that LLMs can follow natural language instructions to solve diverse tasks in a zero-shot setting [64,79]. To rank using LLMs, we infill the patterns above into the instruction template T. An example instruction template can be given as: *" [pattern that contains sequential historical interactions $\mathcal{H}$] [pattern that contains retrieved candidate items $\mathcal{C}$] Please rank these movies by measuring the possibilities that I would like to watch next most, according to my watching history."*.

Parsing the output of LLMs. Note that the output of LLMs is still in natural language text, and we parse the output with heuristic text-matching methods and ground the recommendation results on the specified item set. In detail, we can directly perform efficient substring matching algorithms like KMP [35] between